

Q36 – Partition into K Equal Sum Subsets

1. Aim

To determine whether a given set of positive integers can be divided into **K non-empty subsets having equal sums** using a backtracking algorithm with pruning, and to analyze its correctness and computational complexity.

2. Problem Statement

Given a set of positive integers and an integer **K**, the objective is to divide the elements into **K** subsets such that:

Sum of every subset = Total Sum / K

The problem requires exploring different combinations of elements. Since the number of possible partitions can grow exponentially, a **backtracking approach with pruning** is used to eliminate invalid possibilities as early as possible.

3. Objectives

- Determine whether an equal-sum partition into **K** subsets exists.
 - Generate the subsets when a valid partition exists.
 - Apply backtracking to systematically explore possible partitions.
 - Use pruning to eliminate impossible choices.
 - Define constraints affecting feasibility.
 - Prove the correctness of the algorithm.
 - Analyze time and space complexity.
-

4. Constraints

The following constraints must be satisfied:

1. All input elements must be positive integers.

2. K must be a positive integer.
 3. K cannot be greater than the number of elements.
 4. Every element must belong to exactly one subset.
 5. Each subset must be non-empty.
 6. The total sum must be divisible by K .
 7. The target sum is:
Target Sum = Total Sum / K
 8. No individual element can be greater than the target sum.
 9. Every subset must have exactly the target sum.
-

5. Feasibility Analysis

The constraints strongly influence whether a partition is possible.

Total Sum Divisibility

If:

$$\text{Total Sum} \% K \neq 0$$

then an equal partition is impossible because the target sum would not be an integer.

For example:

Set = [1, 2, 3, 4]

$K = 3$

Total Sum = 10

$$10 \% 3 \neq 0$$

Therefore, no equal-sum partition exists.

Element Greater Than Target

If any element is greater than the target sum, it cannot fit into any subset.

For example:

Set = [8, 2, 3, 3]

$K = 2$

Target = 8

Here, the element 8 can form one subset by itself, but if an element were greater than 8, partitioning would immediately become impossible.

Number of Subsets

If $K > N$, where N is the number of elements, it is impossible to create K non-empty subsets.

6. Backtracking Approach

Backtracking constructs the subsets one element at a time.

For each number, the algorithm tries placing it into each subset.

If placing the number causes the subset sum to exceed the target, that choice is immediately rejected.

If a complete valid partition cannot be obtained, the algorithm removes the element and tries another possibility.

This allows the algorithm to systematically explore the solution space.

7. Pruning Techniques

Several pruning techniques are used to reduce unnecessary computation.

Pruning 1 – Target Overflow

If:

$\text{current subset sum} + \text{element} > \text{target}$

the element is not placed in that subset.

Pruning 2 – Duplicate Subset States

If two subsets currently have the same sum, trying the current element in both produces equivalent states.

Therefore, duplicate states are skipped.

Pruning 3 – Empty Subset Symmetry

If an element is placed into an empty subset and that attempt fails, trying the same element in another empty subset produces an equivalent state.

Therefore, further empty subsets are skipped.

Sorting

The numbers are sorted in descending order.

Large values are placed first, allowing invalid branches to be detected earlier.

8. Algorithm

1. Calculate the total sum of all elements.
 2. Check whether the total sum is divisible by K .
 3. Calculate the target sum.
 4. Check whether any element is greater than the target.
 5. Sort the elements in descending order.
 6. Create K empty subsets.
 7. Select the next element.
 8. Try placing it into each subset.
 9. Reject the placement if the subset sum exceeds the target.
 10. Skip equivalent subset states.
 11. Continue recursively.
 12. If all elements are assigned and every subset has the target sum, return the solution.
 13. Otherwise, backtrack and try another arrangement.
 14. If all possibilities fail, report that partitioning is impossible.
-

9. Pseudocode

START

Read set of numbers and K

Calculate total_sum

IF total_sum % K != 0

 Return "Partition not possible"

END IF

target = total_sum / K

IF K > number of elements

Return "Partition not possible"

END IF

IF maximum element > target

Return "Partition not possible"

END IF

Sort elements in descending order

Create K empty subsets

Create K subset sums initialized to 0

FUNCTION Backtrack(index)

IF index == number of elements

IF all subset sums == target

Return TRUE

ELSE

Return FALSE

END IF

END IF

current = element[index]

FOR each subset i

IF subset sum + current > target

Continue

END IF

IF subset state is duplicate

Continue

END IF

Add current to subset i

Update subset sum

IF Backtrack(index + 1)

Return TRUE

END IF

Remove current
Restore subset sum

IF subset is empty
 Break
END IF

END FOR

Return FALSE

END FUNCTION

Call Backtrack(0)

Display result

STOP

10. Source Code

The program was implemented in **Python** using recursive backtracking.

```

1 # Q36 - Partition into K Equal Sum Subsets
2 # Backtracking with Pruning
3
4 def can_partition(nums, k):
5     total_sum = sum(nums)
6
7     # Constraint 1: Total sum must be divisible by k
8     if total_sum % k != 0:
9         return False, []
10
11     target = total_sum // k
12
13     # There cannot be more subsets than elements
14     if k > len(nums):
15         return False, []
16
17     # If any number is greater than target,
18     # it cannot belong to any valid subset
19     if max(nums) > target:
20         return False, []
21
22     # Sort in descending order for better pruning
23     nums.sort(reverse=True)
24
25     subsets = [[] for _ in range(k)]
26     subset_sums = [0] * k
27
28     def backtrack(index):
29         # All elements have been assigned
30         if index == len(nums):
31             return all(s == target for s in subset_sums)
32
33         current = nums[index]
34         previous_sum = -1
35
36         for i in range(k):
37
38             # Pruning 1:
39             # Do not try identical subset states
40             if subset_sums[i] == previous_sum:
41                 continue
42
43             # Pruning 2:
44             # Do not exceed target sum
45             if subset_sums[i] + current > target:
46                 continue
47
48             previous_sum = subset_sums[i]

```

Figure 1: Python implementation of K equal-sum subset partitioning.

11. Test Case

The following input is used:

Set = [4, 3, 2, 3, 5, 2, 1]

K = 4

Total Sum

$$4 + 3 + 2 + 3 + 5 + 2 + 1 = 20$$

Therefore:

Target Sum = $20 / 4 = 5$

A valid partition is:

Subset 1 = [5]

Subset 2 = [4, 1]

Subset 3 = [3, 2]

Subset 4 = [3, 2]

Every subset has a sum of 5.

12. Execution and Output

The program was executed with the above input.

```
=====
PARTITION INTO K EQUAL SUM SUBSETS
BACKTRACKING + PRUNING
=====

Input Set: [4, 3, 2, 3, 5, 2, 1]
Number of Subsets (K): 4
Total Sum: 20
Target Sum for Each Subset: 5

-----
PARTITION POSSIBLE
-----
Subset 1: [5]      Sum = 5
Subset 2: [4, 1]   Sum = 5
Subset 3: [3, 2]   Sum = 5
Subset 4: [3, 2]   Sum = 5
-----
All subsets have equal sum.
Result: VALID PARTITION
=====
```

Figure 2: Execution showing the valid equal-sum partition.

The output confirms that the input set can be divided into four subsets with equal sums.

13. Correctness

The algorithm is correct because it considers every possible valid placement of each element unless the placement is eliminated by a logically valid pruning condition.

An element is added only when the resulting subset sum does not exceed the target.

When all elements have been assigned, the algorithm accepts the partition only if every subset has exactly the target sum.

Therefore, any partition returned by the algorithm is valid.

If no partition is returned, all non-pruned possibilities have been explored, meaning no valid partition exists.

14. Time Complexity

The problem is computationally difficult and is related to the **NP-complete Partition/3-Partition family**.

In the worst case, the backtracking algorithm may try approximately K^N assignments, where:

- N = number of elements
- K = number of subsets

Therefore, the worst-case time complexity is:

$$O(K^N)$$

Sorting the elements requires:

$$O(N \log N)$$

but the exponential backtracking dominates the total complexity.

Thus, the overall worst-case complexity is:

$$O(K^N)$$

Pruning can significantly reduce the number of states explored in practical cases, although it does not change the worst-case exponential nature of the problem.

15. Space Complexity

The algorithm stores:

- The input elements: $O(N)$
- The K subsets: $O(N)$
- The subset sums: $O(K)$
- Recursive call stack: up to $O(N)$

Therefore, the overall auxiliary space requirement is approximately:

$$O(N + K)$$

16. Advantages

- Finds an actual partition when one exists.
 - Simple recursive structure.
 - Can handle multiple subset constraints.
 - Pruning significantly reduces unnecessary searches.
 - Sorting large elements first improves practical performance.
 - Provides a clear demonstration of backtracking.
-

17. Limitations

- Worst-case running time is exponential.
 - Performance decreases rapidly as the number of elements increases.
 - The algorithm may explore many combinations for difficult inputs.
 - It is not suitable for extremely large instances without additional optimization.
-

18. Effect of Constraints on Feasibility

The constraints allow many impossible cases to be rejected before backtracking begins.

For example:

Total Sum % K $\neq$ 0

immediately proves that a solution does not exist.

Similarly:

Maximum Element > Target Sum

also proves that partitioning is impossible.

During backtracking, the target-sum constraint prevents subsets from exceeding the required value.

Therefore, constraints serve both as **feasibility checks** and **pruning conditions**, reducing unnecessary computation.

19. Applications

The equal-sum partition concept can be applied to:

- Balanced workload distribution.
 - Resource allocation.
 - Task grouping.
 - Equal-capacity scheduling.
 - Data clustering.
 - Manufacturing resource distribution.
 - Partition-based optimization problems.
-

20. Result Analysis

The implemented backtracking algorithm successfully determines whether the given set can be divided into K equal-sum subsets.

For the test case:

Input = [4, 3, 2, 3, 5, 2, 1]

$K = 4$

the total sum is 20, giving a target sum of 5 for each subset.

The algorithm produces a valid partition:

[5]
[4, 1]
[3, 2]
[3, 2]

Each subset has a sum of 5, so the partition is valid.

21. Conclusion

The **Partition into K Equal Sum Subsets** problem was successfully implemented using a **backtracking algorithm with pruning**.

The algorithm first performs feasibility checks using the total sum, number of elements, and target sum. It then recursively constructs subsets while pruning branches that exceed the target or produce duplicate states.

The approach is correct because it explores all relevant possible assignments while eliminating only invalid or equivalent states.

Although pruning improves practical performance, the problem remains computationally challenging because its worst-case time complexity is exponential.

Thus, backtracking provides an effective approach for solving small and moderate-sized instances while clearly demonstrating the computational complexity associated with NP-complete problems.
